PostgreSQL 18 Streaming Replication: A Source-Code-Level Reference (REL_18_STABLE)

TL;DR

- PostgreSQL 18's streaming replication architecture is unchanged in its fundamentals from PG17: WAL is inserted under `WALInsertLocks` (still `NUM_XLOGINSERT_LOCKS == 8`), flushed by `XLogFlush`/walwriter, read and shipped by a per-connection `walsender` (`XLogSendPhysical`), received by a single `walreceiver` (`WalReceiverMain` → `XLogWalRcvProcessMsg`), and replayed serially by the startup process (`PerformWalRecovery` → `ApplyWalRecord`). The single biggest structural bottleneck remains single-threaded redo. (Cybertec's Imran Zaheer, "PostgreSQL Recovery Internals," Dec 30 2025, confirms this against 18.1: `StartupProcessMain → StartupXLOG() → PerformWalRecovery()`, and that standby apply "is actually a continuous form of recovery.")
- The genuinely new PG18 replication item is `idle_replication_slot_timeout` (commit ac0e3313, Nisha Moond/Bharath Rupireddy), which invalidates inactive slots at checkpoint time via `InvalidateObsoleteReplicationSlots(RS_INVAL_IDLE_TIMEOUT, ...)`. AIO (`io_method`) is **read-only** and does **not** touch WAL writes.
- A widely-repeated claim that PG18 doubled WAL write throughput is **false for the shipped release**: the "Get rid of WALBufMappingLock" patch (bc22dc0e) was **reverted** on REL_18_STABLE (commit 2ce6abdf; master c13070a27b) before GA — the revert message states "conditional variables are not suitable for use inside critical sections. If WaitLatch()/WaitEventSetWaitBlock() face postmaster death, they exit, releasing all locks instead of PANIC. In certain situations, this leads to data corruption." `NUM_XLOGINSERT_LOCKS` was not increased. Plan capacity as if PG18 WAL insert scaling equals PG17.

Key Findings

1. **Process model is stable.** The five roles — ordinary backends (WAL inserters), `walwriter`, `walsender` (one per standby), `walreceiver` (one per standby), and the `startup` process (redo) — and their shared-memory control structures (`XLogCtl`, `WalSndCtl`/`WalSnd[]`, `WalRcv`, `ReplicationSlotCtl`, `ProcArray`) are the same as PG17. File paths and function names below are valid on REL_18_STABLE.
2. **PG18's replication-relevant deltas are modest and mostly operational**: `idle_replication_slot_timeout`; `max_active_replication_origins` split out from `max_replication_slots`; logical replication of generated columns; `pg_createsubscriber --all`; several slot-synchronization/failover-slot refinements carried forward from PG17; and a set of 18.x bug fixes touching replication (lag reporting, WAL-receiver survival across timeline switches).
3. **The "2x WAL" headline is a misconception.** It conflates the reverted WALBufMappingLock work (development-branch microbenchmarks of ~4–30%, highest for large records) with the AIO read-path "up to 3×" figure. The latter is from the official PostgreSQL Global Development Group release announcement (Sept 25 2025): AIO support "include sequential scans, bitmap heap scans, and vacuum. Benchmarking has demonstrated performance gains of up to 3x in certain scenarios." Neither describes a shipped PG18 WAL-write improvement.
4. **AIO changes recovery I/O economics slightly, not the redo model.** `io_method=worker` (default) / `io_uring` accelerate *reads*; recovery-side page reads still flow through `recovery_prefetch`/`XLogPrefetcher`, and replay remains one record at a time in a single process.
5. **Synchronous replication internals** center on `SyncRepWaitForLSN` (backends sleep on their own latch in a per-mode `SyncRepQueue[]`), released by `SyncRepReleaseWaiters`/`SyncRepWakeQueue` driven by walsender reply processing.

Details

1. Architecture overview: processes, shared memory, data flow

Primary-side processes.

- **Ordinary backends** construct WAL via `xloginsert.c` (`XLogBeginInsert`, `XLogRegisterData`, `XLogInsert`) and hand a finished record chain to `XLogInsertRecord()` in `src/backend/access/transam/xlog.c`.
- **walwriter** (`src/backend/postmaster/walwriter.c`) opportunistically writes/flushes filled WAL buffers so that commit-time `XLogFlush` does less work.
- **walsender** (`src/backend/replication/walsender.c`): one process per replication connection; physical path runs `WalSndLoop(XLogSendPhysical)`.

Standby-side processes.

- **walreceiver** (`src/backend/replication/walreceiver.c`): a single process, `WalReceiverMain()`, connecting through the `libpqwalreceiver.c` shim (`walrcv_connect`, `walrcv_receive`, `walrcv_send`).
- **startup process** (`src/backend/access/transam/xlogrecovery.c`): `PerformWalRecovery()` drives the redo loop.

Key shared-memory structures.

- `XLogCtlData *XLogCtl` (xlog.c line ~577): WAL buffers, insertion state (`XLogCtlInsert` with `CurrBytePos`/`PrevBytePos`, `WALInsertLocks`), `InsertTimeLineID`.
- `WALInsertLockPadded *WALInsertLocks` (xlog.c ~580): a private pointer to the `NUM_XLOGINSERT_LOCKS`-entry array in `XLogCtl->Insert.WALInsertLocks`. Each lock carries an `insertingAt` LSN advertised via `LWLockUpdateVar`.
- `WalSndCtlData *WalSndCtl` (`walsender_private.h`): array `walsnds[]` of `WalSnd` structs plus the synchronous-replication queues `SyncRepQueue[NUM_SYNC_REP_WAIT_MODE]` and released-LSN array `lsn[]`. Each `WalSnd` holds `pid`, `state`, a spinlock `mutex`, `sentPtr`, `write`, `flush`, `apply`, `sync_standby_priority`, and a `latch`.
- `WalRcvData *WalRcv` (`walreceiver.h`): `walRcvState` (enum `WALRCV_STOPPED/STARTING/STREAMING/WAITING/RESTARTING/STOPPING`), `receiveStart`, `receivedUpto`, `writtenUpto` (atomic), `latestChunkStart`, `lastMsgSendTime`/`lastMsgReceiptTime`, `latch`, `mutex`, `procno`.
- `ReplicationSlotCtl` / `ReplicationSlot` (`slot.c`, `slot.h`): in-memory slot array; each slot's persisted `ReplicationSlotPersistentData` carries `restart_lsn`, `confirmed_flush`, `xmin`, `catalog_xmin`, `invalidated`, `persistency`.
- `ProcArrayStruct` (`procarray.c`): `replication_slot_xmin`, `replication_slot_catalog_xmin`, and the `KnownAssignedXids` machinery used on hot standbys. GitHub

Data flow (text diagram).

```
[backend] XLogInsert → XLogInsertRecord (WALInsertLock) → WAL buffers (XLogCtl)
   → commit: XLogFlush → pg_wal segment (fsync) → SyncRepWaitForLSN (if sync)
        |
        ▼ (walsender reads flushed WAL)
[walsender] XLogSendPhysical → WALRead → CopyData 'w' (XLogData) over libpq (CopyBoth)
        |  ◄── 'r' standby status update / 'h' hot-standby feedback
        ▼
[walreceiver] walrcv_receive → XLogWalRcvProcessMsg → XLogWalRcvWrite (pg_pwrite)
   → XLogWalRcvFlush (updates WalRcv->flushedUpto, writtenUpto) → WakeupRecovery
        |
        ▼
[startup] WaitForWALToBecomeAvailable (XLOG_FROM_STREAM) → ReadRecord
   → ApplyWalRecord → rmgr rm_redo → SetLatch back to walsender/feedback
```

2. End-to-end workflow with functions, files, GUCs

(a) WAL generation and insertion. A backend calls `XLogInsert(rmid, info)` (`xloginsert.c`), which loops calling `XLogInsertRecord()` (`xlog.c`). `XLogInsertRecord` classifies the op (`WalInsertClass`: `WALINSERT_NORMAL`, `WALINSERT_SPECIAL_SWITCH`, `WALINSERT_SPECIAL_CHECKPOINT`), enters a critical section, and acquires a WAL insertion lock. `WALInsertLockAcquire()` picks a lock: `lockToTry = MyProc->pgprocno % NUM_XLOGINSERT_LOCKS` and, on failure to acquire immediately, migrates to another lock so inserters spread across the 8 locks. Space is reserved atomically on `XLogCtl->Insert.CurrBytePos` (protected by the `insertpos_lck` spinlock inside the reservation macros). If a record crosses a page boundary and buffers must be advanced, `AdvanceXLInsertBuffer()` may need to write out older buffers; before acquiring `WALWriteLock` it releases `WALBufMappingLock` and calls `WaitXLogInsertionsToFinish()`, which waits on every insertion lock's `insertingAt` value via `LWLockWaitForVar`. GUCs: `wal_buffers`, `wal_level` (`replica`/`logical` needed for replication), `full_page_writes`, `wal_compression`. Postgresql + 2

> Header comment for `XLogInsertRecord` (xlog.c): "Insert an XLOG record represented by an already-constructed chain of data chunks. This is a low-level routine; ... If 'fpw_lsn' is valid ... If fpw_lsn <= RedoRecPtr, the function does not perform the insertion and returns InvalidXLogRecPtr" — the caller then recomputes full-page images and retries. Postgresql

Wait-event nuance: `LWLock:WALInsert` can mean either "not enough insertion locks" *or* "waiting for another backend to finish copying a record into wal_buffers"; PostgreSQL added distinct wait events precisely because those are operationally different. PostgreSQL

(b) Flush. At commit, `RecordTransactionCommit()` (`xact.c`) calls `XLogFlush(lsn)`. `XLogFlush`/`XLogWrite` (xlog.c) push WAL up to the requested LSN to disk under `WALWriteLock`, honoring `wal_sync_method`. `walwriter` performs background flushing on `wal_writer_delay`/`wal_writer_flush_after`. GUCs: `synchronous_commit`, `commit_delay`, `commit_siblings`, `fsync`, `wal_sync_method`, `wal_writer_delay`.

(c) walsender startup & protocol handshake. A replication connection is a libpq connection with `replication=true` (physical) or `replication=database` (logical) in the startup packet; only the simple query protocol is allowed. `exec_replication_command()` (walsender.c) parses replication grammar (`repl_gram.y`/`repl_scanner.l`) and dispatches `IDENTIFY_SYSTEM`, `CREATE_REPLICATION_SLOT`,

`START_REPLICATION`, `BASE_BACKUP`, `TIMELINE_HISTORY`, etc. For physical streaming, `StartReplication()` sends a `CopyBothResponse` (`'W'`), sets `WalSndSetState(WALSNDSTATE_CATCHUP)`, validates that the requested start point does not exceed the server flush position ("requested starting point %X/%08X is ahead of the WAL flush position of this server"), records `sentPtr` in the shared `WalSnd`, calls `SyncRepInitConfig()`, and enters `WalSndLoop(XLogSendPhysical)`. Access control uses `pg_hba.conf` with the special `replication` database keyword. PostgreSQL Urho3D

(d) walsender main loop / sending WAL. `XLogSendPhysical()` (walsender.c) first calls `WalSndWaitForWal(targetPagePtr + reqLen)` to ensure enough WAL is flushed (for a cascading standby it uses `GetStandbyFlushRecPtr()` instead of `GetFlushRecPtr()`), then reads with `WALRead()` and frames an `XLogData` `'w'` message via `pq_putmessage_noblock('d', ...)`. Keepalives are sent by `WalSndKeepalive()` (`'k'`). `WalSndLoop` also runs `ProcessRepliesIfAny()` to consume `'r'`/`'h'` messages and, on physical primaries, `SyncRepReleaseWaiters()`. Timeout handling: if `wal_sender_timeout` elapses with no reply, "terminating walsender process due to replication timeout". GUCs: `max_wal_senders` (default 10; 0 disables replication), `wal_sender_timeout`, `wal_keep_size`, `max_slot_wal_keep_size`, `synchronous_standby_names`. Postgresql + 2

(e) Network protocol (byte-level). WAL travels inside libpq `CopyData` (`'d'`) messages; the first payload byte selects the sub-message (documented in "54.4. Streaming Replication Protocol"):

- `'w'` **XLogData (server→standby):** byte `'w'`; Int64 = starting point of WAL in this message; Int64 = current end of WAL on server; Int64 = server sendtime (μs since 2000-01-01); then the raw WAL byte stream. "A single WAL record is never split across two XLogData messages" — except that a record already split across a page boundary by continuation records may be split at that page boundary. PostgreSQL PostgreSQL
- `'k'` **Primary keepalive (server→standby):** byte `'k'`; Int64 current end of WAL; Int64 sendtime; Byte1 replyRequested (1 ⇒ reply ASAP to avoid timeout). PostgreSQL
- `'r'` **Standby status update (standby→server):** byte `'r'`; Int64 last WAL byte+1 written to disk; Int64 last WAL byte+1 flushed; Int64 last WAL byte+1 applied; Int64 client sendtime; Byte1 replyRequested. PostgreSQL
- `'h'` **Hot-standby feedback (standby→server):** byte `'h'`; Int64 client sendtime; Int32 xmin (global xmin excluding slot catalog_xmin); Int32 xmin epoch; Int32 catalog_xmin; Int32 catalog_xmin epoch. If xmin and catalog_xmin are both 0, feedback is being disabled on this connection. PostgreSQL

(f) walreceiver write/flush. `WalReceiverMain()` marks itself running in `WalRcv`, connects (`walrcv_connect`), issues `START_REPLICATION`, then loops: `walrcv_receive(NAPTIME_PER_CYCLE, &buf)` (100 ms max sleep), and for each message `XLogWalRcvProcessMsg(buf[0], &buf[1], len-1)`. `'w'` → `XLogWalRcvWrite()` (`pg_pwrite` into the current segment, updating `writtenUpto`; counts `IOOBJECT_WAL`/`IOOP_WRITE` stats when `track_wal_io_timing`); flush via `XLogWalRcvFlush()` which advances `WalRcv->flushedUpto` and calls `WakeupRecovery()`. `'k'` with replyRequested triggers `XLogWalRcvSendReply()`. Feedback: `XLogWalRcvSendHSFeedback()` sends `'h'` when `hot_standby_feedback=on`, computing xmin via `GetOldestXmin()`. GUCs: `primary_conninfo`, `primary_slot_name`, `wal_receiver_timeout` (default 60s), `wal_receiver_status_interval`, `hot_standby_feedback`, `wal_retrieve_retry_interval`. LESIA Postgresql

(g) Startup process redo. `PerformWalRecovery()` runs the main redo loop: `ApplyWalRecord(xlogreader, record, &replayTLI)` then `record = ReadRecord(xlogprefetcher, LOG, false, replayTLI)` until NULL. `ApplyWalRecord` dispatches to the resource manager's `rm_redo` via `GetRmgr()`, updating `XLogRecoveryCtl->lastReplayedEndRecPtr`/`lastReplayedTLI`, and periodically calls `WalSndWakeup()` (to push cascaded WAL) and `WalRcvForceReply()`. WAL acquisition is the state machine `WaitForWALToBecomeAvailable()` (xlogrecovery.c) cycling `XLOG_FROM_ARCHIVE` → `XLOG_FROM_PG_WAL` → `XLOG_FROM_STREAM`. Reads are fed through `XLogPrefetcher` (xlogprefetcher.c), described in-tree as "a drop-in replacement for an XLogReader that tries to minimize IO stalls by looking ahead in the WAL." GUCs: `recovery_min_apply_delay`, `recovery_prefetch`, `max_standby_streaming_delay`, `max_standby_archive_delay`, `hot_standby`. Pgedge + 2

(h) Synchronous wait/release. See section 3.

3. Synchronous replication internals

`synchronous_standby_names` is parsed by `syncrep_gram.y`/`syncrep_scanner.l` into `SyncRepConfig` (`SyncRepConfigData` with `syncrep_method` = `SYNC_REP_PRIORITY` or `SYNC_REP_QUORUM`, and `num_sync`). `synchronous_commit` maps to a wait mode in `SyncRepWaitMode`: `remote_write` → `SYNC_REP_WAIT_WRITE`, `on` → `SYNC_REP_WAIT_FLUSH`, `remote_apply` → `SYNC_REP_WAIT_APPLY`; `local`/`off` → `SYNC_REP_NO_WAIT`.

Backend side — `SyncRepWaitForLSN(XLogRecPtr lsn, bool commit)` (syncrep.c ~149), called from `RecordTransactionCommit`, `EndPrepare`, `RecordTransactionAbortPrepared`, `RecordTransactionCommitPrepared`:

- Fast exit if `!SyncRepRequested() || !SyncStandbysDefined()`. ltworkman
- Under `SyncRepLock`, if `WalSndCtl->lsn[mode]` already ⩾ our LSN, return (already satisfied). Alibaba Cloud

- Otherwise set `MyProc->waitLSN = lsn`, `syncRepState = SYNC_REP_WAITING`, and `SyncRepQueueInsert(mode)` which inserts the PGPROC into `WalSndCtl->SyncRepQueue[mode]` **ordered by LSN** (`dlist` with `PGPROC.syncRepLinks`).
- Loop: `ResetLatch(MyLatch)`, test `syncRepState`, `WaitLatch(..., WL_LATCH_SET | WL_POSTMASTER_DEATH, ...)` until state becomes `SYNC_REP_WAIT_COMPLETE`.

Header comment: "Initially backends start in state SYNC_REP_NOT_WAITING and then change that state to SYNC_REP_WAITING before adding ourselves to the wait queue. During SyncRepWakeQueue() a WALSender changes the state to SYNC_REP_WAIT_COMPLETE once replication is confirmed. This backend then resets its state to SYNC_REP_NOT_WAITING." Itworkman

Walsender side — `SyncRepReleaseWaiters()` (syncrep.c ~484): called after each standby reply. It determines whether this walsender is (one of) the synchronous standbys via `SyncRepGetSyncRecPtr`/`SyncRepGetCandidateStandbys` (priority vs quorum via `SyncRepGetNthLatestSyncRecPtr`/`SyncRepGetOldestSyncRecPtr`), then advances the released LSNs and wakes queues:

```
if (walsndctl->lsn[SYNC_REP_WAIT_WRITE] < MyWalSnd->write) {
    walsndctl->lsn[SYNC_REP_WAIT_WRITE] = MyWalSnd->write;
    numwrite = SyncRepWakeQueue(false, SYNC_REP_WAIT_WRITE);
}
if (walsndctl->lsn[SYNC_REP_WAIT_FLUSH] < MyWalSnd->flush) { ... SyncRepWakeQueue(false, SYNC_REP_WAIT_FLUSH); }
```

`SyncRepWakeQueue(bool all, int mode)` walks the LSN-ordered queue; for each PGPROC with `waitLSN <= lsn[mode]` it detaches the node (`dlist_delete_thoroughly`), issues `pg_write_barrier()`, sets `syncRepState = SYNC_REP_WAIT_COMPLETE`, and `SetLatch(&proc->procLatch)`. The policy comment: "This implements a simple policy of first-valid-standby-releases-waiter." Note the wait is genuinely blocking — the backend spins in a `for(;;)` latch loop and does no other work until woken. Also note `remote_apply` (flush-then-apply) can mean a transaction is flushed on the standby but not yet visible during a recovery conflict. Alibaba Cloud PostgreSQL

4. Replication slots internals

Creation/acquire: `ReplicationSlotCreate()`, `ReplicationSlotAcquire()`, `ReplicationSlotRelease()` (slot.c). Physical slots have `data.database == InvalidOid` (`SlotIsPhysical`). On-disk state lives in `pg_replication_slots/<name>/state` and is written by `SaveSlotToPath()` (fsync'd), restored by `RestoreSlotFromDisk()`. `restart_lsn` is the oldest WAL the slot still needs; `confirmed_flush` (logical) tracks confirmed decode progress. The Mail Archive

WAL retention: `KeepLogSeg()` in `CreateCheckPoint()` (xlog.c) computes the oldest segment to keep from `restart_lsn`, `wal_keep_size`, and `max_slot_wal_keep_size`, then `InvalidateObsoleteReplicationSlots()` runs. Postgresql

Invalidation: `InvalidatePossiblyObsoleteSlot()` and `DetermineSlotInvalidationCause()` (slot.c) evaluate causes: `RS_INVAL_WAL_REMOVED` (restart_lsn older than the oldest retained segment), `RS_INVAL_HORIZON` (conflict with recovery / xmin horizon), `RS_INVAL_WAL_LEVEL`, and PG18's new `RS_INVAL_IDLE_TIMEOUT`. Since PG16 (commit 818fefd8fd4), the decision uses the `initial_restart_lsn`/initial xmin captured at the start of the check (to avoid a race where an active slot's restart_lsn advances after the mutex is released), and it terminates the holding backend before marking the slot obsolete. On `RS_INVAL_WAL_REMOVED` the code sets `data.restart_lsn = InvalidXLogRecPtr` and records `invalidated_at`. The Mail Archive The Mail Archive

PG18 `idle_replication_slot_timeout` (commit ac0e3313): invalidates a slot inactive longer than the configured duration, measured from `pg_replication_slots.inactive_since`. Default 0 (disabled); `sighup`, postgresql.conf only. Invalidation is applied at checkpoint (`InvalidateObsoleteReplicationSlots(RS_INVAL_WAL_REMOVED | RS_INVAL_IDLE_TIMEOUT, ...)`), so there can be lag up to `checkpoint_timeout` unless a manual `CHECKPOINT` is issued. Documented caveat: the value rounds to whole minutes, so `30s` rounds to 0 and effectively disables it. Not applicable to slots that don't reserve WAL, nor to synced standby slots (`synced=true`), which are always considered inactive because they don't perform logical decoding.

5. Failure and edge cases

- **Timeline switches / promotion.** `pg_promote()` (xlogfuncs.c) or a promote signal file sets `SharedPromoteIsTriggered`; `CheckForStandbyTrigger()` (xlogrecovery.c) detects it. On promotion the server ends recovery, writes an end-of-recovery/checkpoint record, bumps the timeline, and streaming clients follow via the `TIMELINE_HISTORY`/timeline-switch result set that `StartReplication` emits when `sendTimeLineIsHistoric`.
- **WAL-receiver survival across stream→archive transitions (PG18 fix).** A REL_18_STABLE bug fix (commit a14201073965…, "Fix unconditional WAL receiver shutdown during stream-archive transition"; Xuneng Zhou, reviewed by Noah Misch/Michael Paquier, back-patched through 13) stops the walreceiver from being needlessly killed and restarted on a timeline change, which had confused status monitoring. It ensures `WaitForWALToBecomeAvailable()` is not reached while the receiver is `WALRCV_STOPPING`. The Mail Archive
- **`InstallXLogFileSegmentActive` assertion race.** Historically, switching from `XLOG_FROM_STREAM` back to archive without shutting down the receiver tripped `!IsInstallXLogFileSegmentActive()`; the fix resets the flag when entering archive recovery. Postgres Professional

- **Slot invalidation vs slot copy.** A slot invalidated by `RS_INVAL_IDLE_TIMEOUT` keeps a non-invalid `restart_lsn` in memory, which historically allowed copying a slot whose WAL was already removed (assertion failure in `copy_replication_slot`); this class of races was addressed by restricting copying of invalidated slots. The Mail Archive
- **Newly-created slot invalidated by concurrent checkpoint.** An active area (fixed in back branches; discussed for HEAD) where a slot's `restart_lsn` is assigned while a checkpoint concurrently advances the redo pointer, risking immediate invalidation. The Mail Archive
- **Standby falling behind → WAL removal.** Without a slot, if a standby lags beyond `wal_keep_size`, the primary may recycle a needed segment and terminate the connection ("requested WAL segment … has already been removed"); the standby can still recover from archive if `restore_command` is set.

6. Hot standby: conflicts and feedback

On the standby, replay of cleanup/vacuum records can conflict with running queries. `ResolveRecoveryConflictWithSnapshot()` (standby.c) computes conflicting virtual xids and signals them (`PROCSIG_RECOVERY_CONFLICT_SNAPSHOT`), subject to `max_standby_streaming_delay`/`max_standby_archive_delay`. Standby snapshots are built from `KnownAssignedXids` in `procarray.c` (populated from WAL running-xact and xact-assignment records). `hot_standby_feedback=on` makes the standby report its oldest xmin via the `'h'` message; the primary's walsender stores it in `MyProc->xmin`, so `GetOldestXmin()`/vacuum on the primary will not remove tuples still visible on the standby — at the cost of potential bloat. Note `vacuum_defer_cleanup_age` was **removed in PG16** in favor of `hot_standby_feedback`; do not expect it in PG18. Feedback only protects the standby's xmin if the walsender runs continuously; using a physical slot (`primary_slot_name`) persists xmin/catalog_xmin across disconnects. CYBERTEC PostgreSQL + 2

7. Cascading replication

A standby can itself run walsenders (`am_cascading_walsender`). `XLogSendPhysical` uses `GetStandbyFlushRecPtr()` for the send limit; `WalSndWakeup()` from the redo loop nudges cascaded senders as records are replayed. `hot_standby_feedback` and status updates propagate up the chain. Postgresql

8. Monitoring hooks and the shared memory they read

- **`pg_stat_replication`** (one row per active walsender; on the primary): `pg_stat_get_wal_senders()` reads the `WalSnd[]` array. Columns: `pid`, `usename`, `application_name`, `client_addr`, `backend_xmin`, `state` (`startup`/`catchup`/`streaming`/`backup`), `sent_lsn`, `write_lsn`, `flush_lsn`, `replay_lsn`, `write_lag`, `flush_lag`, `replay_lag` (intervals from the walsender `LagTracker`), `sync_priority`, `sync_state` (`async`/`potential`/`sync`/`quorum`). Byte lag = `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`. Per the docs, the lag columns measure "the time taken for recent WAL to be written, flushed and replayed and for the sender to know about it," and revert to NULL on a fully-replayed idle system. PG18.x fixed premature-NULL and stalled lag reporting bugs (18.1: "Fix premature NULL lag reporting in pg_stat_replication"). Postgresql
- **`pg_stat_wal_receiver`** (one row on the standby): `pg_stat_get_wal_receiver()` reads `WalRcv`. Columns: `pid`, `status`, `receive_start_lsn`, `receive_start_tli`, `written_lsn`, `flushed_lsn`, `received_tli`, `last_msg_send_time`, `last_msg_receipt_time`, `latest_end_lsn`, `latest_end_time`, `slot_name`, `sender_host`, `conninfo`.
- **`pg_replication_slots`**: reads the slot array — `restart_lsn`, `confirmed_flush_lsn`, `wal_status` (`reserved`/`extended`/`unreserved`/`lost`), `safe_wal_size`, `active`, `inactive_since`, `invalidation_reason` (e.g., `idle_timeout`, `wal_removed`), `synced`, `failover`, `conflicting`.
- **`pg_stat_replication_slots`**: logical slot decode stats (spill/stream counters).
- **`pg_stat_recovery_prefetch`**: `wal_distance`, `block_distance`, `io_depth` (current) plus cumulative prefetch counters.
- Standby SQL helpers: `pg_is_in_recovery()`, `pg_last_wal_receive_lsn()`, `pg_last_wal_replay_lsn()`, `pg_last_xact_replay_timestamp()`, `pg_wal_replay_pause()/resume()`.

9. PG18-specific changes relevant to replication and WAL

- **`idle_replication_slot_timeout`** (commit ac0e3313; Nisha Moond, Bharath Rupireddy) — the marquee replication-slot addition. Mechanism as in §4.
- **`max_active_replication_origins`** (Euler Taveira) — decouples replication-origin count from `max_replication_slots`.
- **Logical replication of generated columns** (commits 745217a0, 7054186c) — physical/streaming path unaffected but part of the release's replication story.
- **`pg_createsubscriber --all`** / **`--clean`** / **`--enable-two-phase`**; **`pg_recvlogical --enable-failover`** — tooling.
- **Asynchronous I/O (AIO), `io_method` (initial commit da722699)** — default `worker`; options `sync`, `worker`, `io_uring`. **AIO is read-only in PG18**: sequential scans, bitmap heap scans, VACUUM/ANALYZE. Multiple named sources confirm WAL writes are excluded — Neon: "Read operations only: Write operations, including WAL writes, remain synchronous"; pgEdge: "AIO is currently only for read operations and not for write and WAL operations." Recovery-side reads still use `recovery_prefetch`/`XLogPrefetcher` (PG15+). Per the

PG18 release notes (E.1), the default of `effective_io_concurrency` and `maintenance_io_concurrency` was raised to 16 (from 1) — "Increase server variables effective_io_concurrency's and maintenance_io_concurrency's default values to 16 (Melanie Plageman)... This more accurately reflects modern hardware" (commit ff79b5b2aba02d720f9b7fff644dd50ce07b8c6e) — which can modestly help prefetch-driven recovery I/O concurrency.

- **The reverted WAL-concurrency work (important correction).** The development-branch commit bc22dc0e "Get rid of WALBufMappingLock" (Yura Sokolov / Alexander Korotkov, 2025-04-02) allowed concurrent WAL-buffer initialization via atomic `InitializedUpTo` advancement plus per-buffer condition variables. It was **reverted on REL_18_STABLE** by commit 2ce6abdf (master: c13070a27b; Alexander Korotkov, 2025-08-22, back-patched through 18) with the message: "Revert 'Get rid of WALBufMappingLock' ... It appears that conditional variables are not suitable for use inside critical sections. If WaitLatch()/WaitEventSetWaitBlock() face postmaster death, they exit, releasing all locks instead of PANIC. In certain situations, this leads to data corruption." (The failure manifested as visibility-map corruption on standbys.) Consequently `WALBufMappingLock` **still exists** in shipped PG18 and `NUM_XLOGINSERT_LOCKS` **remains 8**. Treat PG18 WAL-insert concurrency as equal to PG17.
- **18.x replication bug fixes** (release notes): survive WAL-receiver across stream→archive timeline switch (18.4/backpatch); fix `pg_stat_replication` lag reporting (18.1); fix parallel-apply lock-timeout signal collision; ensure FSM changes are persisted during recovery when checksums are enabled (a redo/standby correctness fix — Alexey Makhmutov); avoid indefinite walsender wait at shutdown of a logical publisher; prevent stuck slotsync workers from blocking promotion (Nisha Moond, Ajin Cherian).

10. Throughput / latency benchmarks (with sources; gaps noted)

Benchmark data specific to PG18 streaming replication is scarce; the mechanics are unchanged from PG17, so PG16/17-era numbers remain representative.

- **Single-threaded replay ceiling (Cybertec, Tomas Vondra).** On a 24-thread workstation, 30 min of `pgbench` at `synchronous_commit=off` produced ~70 GB of WAL / 66M transactions at ~36.6k tps on the primary; replaying that backup took 372 s = **~177k tps of pure redo**. Interpretation: a single replay core can outpace a moderately-loaded primary, but a heavily-parallel write primary (many backends) can structurally outrun a single-threaded standby — the lag then grows without bound. pgbench is write-heavy, so real-world crossover is higher. CYBERTEC PostgreSQL
- **Synchronous vs asynchronous overhead (EDB, r5.2xlarge EC2, single AZ, two 10k-IOPS io2 volumes, ~150 GB pgbench db, 10 ms injected inter-node RTT).** With modern code, `remote_write` reached ~92% of `local` throughput at 40 clients and parity by ~80 clients; the gap between sync levels closes as concurrency rises because commit acks (unlike storage I/O) are non-destructive and pipeline well. EnterpriseDB EnterpriseDB
- **Sync-level spread (EDB).** In an older pgbench comparison, `synchronous_commit=off` delivered >2× the throughput of `remote_apply`, with `on` in between — the canonical ordering `off > local > remote_write > on > remote_apply`, magnitude highly workload/network dependent. EnterpriseDB
- **RTT sensitivity (pgconf.de 2025, "Myths and Truths about Synchronous Replication").** Emulated-latency pgbench (`synchronous_standby_names='FIRST|ANY 1 (*)'`, `synchronous_commit=on`) shows TPS scaling inversely with RTT and improving with client count — reinforcing that synchronous latency cost is hidden by concurrency. Postgresql
- **AIO read gains (not WAL).** The official PostgreSQL 18 release announcement (Sept 25 2025) cites "performance gains of up to 3x in certain scenarios" from AIO for sequential scans, bitmap heap scans, and vacuum "when reading from storage." Independent PlanetScale/Elestio benchmarks on ~300 GB datasets found cold-cache reads with `io_uring` dropped from ~15 s to ~5.7 s (roughly 3×), with warm-cache scenarios seeing ~20–25% gains. These do **not** apply to WAL shipping or synchronous-commit latency.

Gaps: I found no rigorous, published PG18-vs-PG17 benchmark isolating streaming-replication throughput or replay rate; no public numbers on `idle_replication_slot_timeout` overhead (it is checkpoint-time and negligible); and the frequently-cited "2× WAL write throughput" figure is not attributable to any shipped PG18 feature (see §9).

Recommendations

1. **Capacity-plan WAL and replication as PG17-equivalent.** Because bc22dc0e was reverted and `NUM_XLOGINSERT_LOCKS` stays at 8, do not budget for a PG18 WAL-insert concurrency win. If you see `LWLock:WALInsert` waits, first distinguish true lock contention from WAL-buffer copy waits (use the distinct wait events), raise `wal_buffers`, and reduce record volume (`wal_compression`, fewer full-page writes via checkpoint tuning) before assuming lock scaling. *Threshold to escalate:* sustained `LWLock:WALInsert` in the top wait events with WAL generation near your storage's sequential write ceiling.
2. **Adopt `idle_replication_slot_timeout` deliberately.** Set it in whole-minute units (e.g., `'24h'` or `'72h'`) as a safety net against WAL bloat from abandoned slots; remember invalidation only fires at checkpoint, and it does not apply to synced standby slots. For production CDC, many teams still prefer `0` (disabled) plus active monitoring of `pg_replication_slots` (`active=false`, growing `restart_lsn` gap) to avoid surprise drops. *Threshold to change:* if `pg_wal` growth from an inactive slot threatens disk, enable it or set `max_slot_wal_keep_size`.

3. **Treat single-threaded replay as the primary scaling risk.** Monitor `replay_lag` and `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`. If replay lag grows under write bursts while the standby's startup process is CPU-bound near 100%, you are at the redo ceiling — mitigations are `recovery_prefetch=try` (default) with a higher `effective_io_concurrency`/`maintenance_io_concurrency` (now 16 by default) for I/O-bound replay, faster standby storage/CPU, splitting write workloads, or offloading reads to more replicas. Prefetch cannot help a CPU-bound redo. `Pgedge` `Pgedge`
4. **Choose `synchronous_commit` by RTT and concurrency, not by fear.** For low-RTT same-AZ standbys, `on` (flush) costs little at realistic client counts; `remote_write` buys latency at the cost of durability-on-standby; reserve `remote_apply` for read-your-writes-on-standby requirements and expect the largest penalty. Use `ANY n (...)` quorum sets for availability. Benchmark with your own pgbench + injected RTT.
5. **AIO tuning is orthogonal to replication.** Keep `io_method=worker` (default) unless you have validated `io_uring` on a modern kernel; expect gains on read/scan/VACUUM only. Do not expect AIO to improve WAL flush, synchronous-commit latency, or replay throughput.
6. **Apply the latest 18.x minor.** Several replication-correctness and monitoring fixes landed in 18.1–18.4 (lag reporting, WAL-receiver survival on timeline switch, FSM persistence with checksums, slotsync/promotion). Given data checksums are on by default in PG18-initialized clusters, the FSM-persistence-during-recovery fix is especially relevant to standbys.

Caveats

- **Line numbers drift.** Function locations cited (e.g., `SyncRepWaitForLSN` ~syncrep.c:149, `XLogWalRcvProcessMsg` ~walreceiver.c:884/890, `XLogCtl` ~xlog.c:577) are approximate and shift across minor releases and doxygen snapshots; verify against the exact REL_18_STABLE tag you build. Some quoted snippets originate from doxygen/mirror captures that may reflect master rather than REL_18_STABLE; the semantics described match REL_18_STABLE but exact wording and line numbers should be confirmed in-tree.
- **The "2× WAL throughput" claim is corrected here as not applicable to shipped PG18** based on the revert (2ce6abdf / c13070a27b). If a future minor or PG19 re-lands a corrected version of the concurrent-buffer-init work, this conclusion changes.
- **Benchmark numbers are environment-specific** (instance type, storage, RTT, pgbench scale/mix) and are cited to illustrate orders of magnitude, not to predict your throughput. The single most portable finding is the *shape* of the results (redo is single-threaded; sync overhead shrinks with concurrency), not the absolute tps.
- **Logical replication** internals (pgoutput, logical decoding, `START_REPLICATION SLOT ... LOGICAL`, protocol versions 1–4) are referenced only where they intersect physical streaming; a full logical-decoding treatment is out of scope.
- Protocol byte layouts are quoted from the official "Streaming Replication Protocol" chapter (stable across recent majors); confirm against the PG18 docs for any edge message you depend on.